

UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA

TEC508 - Computação de Alto Desempenho

Prof. Angelo Duarte

Paralelismo com Memória Compartilhada

Inversão de Matriz por Eliminação de Gauss-Jordan: Análise Comparativa de Desempenho entre Implementações Compiladas e Interpretadas, Seriais e Paralelas

Joanderson Santos

Feira de Santana, Bahia

2026

1. Introdução

1.1 Objetivo

Este trabalho tem como objetivo proporcionar uma compreensão prática e aprofundada das diferenças de desempenho entre linguagens compiladas e interpretadas no contexto de execução paralela em arquiteturas multicore. Para isso, um mesmo algoritmo; inversão de matriz por eliminação de Gauss-Jordan com pivotamento parcial; foi implementado em cinco versões distintas: C Serial, C com OpenMP, Python Serial, Python com multithreading e Python com multiprocessing. Cada versão foi submetida a profiling detalhado (gprof, perf, cProfile, Valgrind, strace) e a medição sistemática de tempo de execução, permitindo identificar hotspots, quantificar métricas de hardware (cache-misses, branch mispredictions, IPC) e comparar objetivamente o overhead do interpretador Python frente ao código compilado em C, bem como a eficiência real de cada modelo de paralelização.

1.2 Três Distinções Conceituais

Antes de apresentar os resultados, é importante separar com clareza três distinções conceituais que, por usarem terminologia semelhante, são facilmente confundidas ao longo da análise:

a) CPU-bound vs. I/O-bound

Classifica se o tempo total de execução é dominado por processamento (CPU) ou por espera de operações de entrada/saída (disco, rede). É avaliada pela proporção entre tempo de usuário/sistema e tempo total, e pela ausência ou presença de page faults maiores. Aplica-se a todas as cinco versões deste trabalho (Seção 3).

b) Compute-bound vs. Memory-bound

Distinção mais fina, aplicável dentro do tempo de CPU: mesmo um programa inteiramente CPU-bound pode ter seu desempenho limitado não pela capacidade de cálculo do processador, mas pela velocidade com que os dados chegam da memória RAM até as unidades de execução. Um algoritmo é memory-bound quando sua intensidade aritmética (razão entre operações de ponto flutuante e bytes acessados) é baixa. Esta distinção é central para explicar por que o C com OpenMP não escala como a Lei de Amdahl prevê (Seção 6.3).

c) Memória Compartilhada vs. Memória Distribuída

Refere-se ao modelo de execução paralela, não a uma característica de desempenho: threads (e, por extensão, o OpenMP) compartilham o mesmo espaço de endereçamento do processo; qualquer variável é diretamente visível a todas as threads. Processos separados (multiprocessing) têm espaços de memória isolados por padrão, exigindo comunicação explícita entre processos (IPC) para trocar dados. Esta é a distinção que dá nome ao próprio trabalho, e é discutida em detalhe nas Seções 6 e 7.

1.3 Ambiente de Teste

Todas as medições foram realizadas na máquina real do autor, conforme exigido pelo enunciado, com as seguintes características:

Componente	Especificação
------------	---------------

Processador	Intel Core i7-3770 @ 3,90 GHz
Núcleos	4 físicos / 8 lógicos (Hyper-Threading)
Memória RAM	16 GiB (DDR3)
Sistema Operacional	Linux (Ubuntu 24.04.2 LTS)

O mapeamento de Hyper-Threading do processador, obtido via `lscpu -e`, mostrou que as CPUs lógicas formam quatro pares que compartilham fisicamente o mesmo núcleo: (0,4), (1,5), (2,6) e (3,7). Este mapeamento é fundamental para a análise de causa raiz apresentada na Seção 6.3, pois threads que caem no mesmo par físico disputam a mesma unidade de ponto flutuante e as mesmas caches L1/L2.

2. Metodologia

2.1 O Algoritmo

O algoritmo escolhido; eliminação de Gauss-Jordan com pivotamento parcial; calcula a inversa de uma matriz quadrada A ($N \times N$, precisão double) transformando a matriz aumentada $[A \mid I]$ em $[I \mid A^{-1}]$ por meio de N etapas de eliminação. Em cada etapa k , localiza-se o elemento de maior valor absoluto na coluna k (pivotamento parcial, para estabilidade numérica), normaliza-se a linha k , e subtrai-se um múltiplo dela de todas as demais linhas. O núcleo computacional (hotspot) é a operação $M[i][j] -= \text{factor} * M[k][j]$, executada $O(N^3)$ vezes.

Esta operação tem intensidade aritmética muito baixa: para cada elemento atualizado, são realizadas duas leituras de memória e uma escrita (24 bytes, assumindo double) para apenas duas operações de ponto flutuante (uma multiplicação e uma subtração). Essa característica; poucas operações para muito tráfego de memória; é o que torna o algoritmo memory-bound, como discutido na Seção 6.3.

2.2 Calibração do Volume de Dados

O enunciado exige tempo de execução mínimo de 20 segundos para a versão C compilada com `-O2`. Testes exploratórios em tamanhos crescentes de matriz mostraram crescimento próximo de $O(N^3)$, levando à calibração de $N = 2500$, que produziu 29,82 segundos de wall-clock na versão C Serial; acima do mínimo exigido, mas ainda praticável para as versões Python (cuja execução serial consome cerca de 32 minutos com este mesmo N).

2.3 Verificação de Corretude

A corretude de cada implementação foi verificada computando o resíduo de Frobenius $\|A \cdot A^{-1} - I\|$, que deve ser próximo de zero. Todas as cinco versões produziram resíduos da ordem de 10^{-14} (por exemplo, $9,12 \times 10^{-14}$ para $N=2500$), consistente com o erro de arredondamento esperado de aritmética de ponto flutuante em precisão dupla; não exatamente zero, mas suficientemente pequeno para confirmar que as cinco implementações calculam o mesmo resultado matemático.

2.4 Ferramentas de Profiling por Versão

Versão	Ferramentas aplicadas
--------	-----------------------

C Serial	/usr/bin/time -v, gprof, perf stat, perf record, Callgrind, Cachegrind, strace
C + OpenMP	/usr/bin/time -v, perf stat, perf record
Python Serial	/usr/bin/time -v, cProfile, perf stat, strace
Python Threading	/usr/bin/time -v, cProfile, perf stat, perf record
Python Multiprocessing	/usr/bin/time -v, cProfile, perf stat, perf record*

* O perf record da versão Multiprocessing gerou um arquivo de aproximadamente 9 GB, computacionalmente impraticável de processar (ver limitação discutida na Seção 7.4). A análise dessa versão apoia-se em perf stat e cProfile, que forneceram evidência suficiente e convergente.

3. Medição de Tempo; As 5 Versões (/usr/bin/time -v)

Para cada versão, o tempo de execução foi medido com /usr/bin/time -v, que fornece wall-clock, tempo de usuário e sistema, percentual de CPU utilizado, memória residente máxima (RSS), page faults e trocas de contexto. A Tabela 1 apresenta os resultados para a configuração de referência de cada versão (C OpenMP, Python Threading e Python Multiprocessing na configuração de 4 threads/processos, usada como referência ao longo deste relatório por ser o ponto de comparação direta com a versão serial em núcleos físicos, sem envolver Hyper-Threading).

Versão	Wall-clock	User	Sys	%CPU	RSS	Minor PF	Ctx (Vol/Invol)
C Serial	29,82 s	29,71 s	0,10 s	99%	192,6 MB	56.656	1 / 305
C + OpenMP (4t)	24,44 s	96,79 s	0,60 s	398%	192,9 MB	56.672	55 / 776
Python Serial	31min 45s	1902,12 s	1,66 s	99%	918,5 MB	270.357	1 / 35.076
Python Threading (4t)	1h 11min 42s	4339,18 s	47,17 s	101%	919,6 MB	280.530	3.746.390 / 40.662
Python Multiproc. (4p)	32min 51s	7705,42 s	2,54 s	390%	592,2 MB	270.452	82.562 / 42.486

Tabela 1; Medição /usr/bin/time -v das cinco versões (configuração de referência: 4 threads/processos para as versões paralelas).

3.1 Classificação CPU-bound vs. I/O-bound

Todas as cinco versões classificam-se como fortemente CPU-bound. Dois indicadores sustentam essa classificação de forma consistente: o tempo de sistema (sys) nunca ultrapassa 2% do tempo total em nenhuma versão, e o número de page faults maiores (que exigiriam I/O de disco) é zero em todas; não incluído na Tabela 1 por ser uniformemente nulo. Isso confirma que o tempo de execução é integralmente dominado por processamento, sem gargalo de leitura/escrita em disco.

Um padrão que antecipa a análise da Seção 7 já é visível nesta tabela: os context switches involuntários da versão Threading (40.662) são muito superiores aos das demais versões, mesmo com apenas 4 threads; sinal precoce da disputa pela trava global do interpretador (GIL), tratada em detalhe adiante.

4. Profiling; C Serial

4.1 gprof: Identificação do Hotspot

A compilação exigida para este trabalho usa a flag -O2 (diferentemente de um profiling anterior realizado com -O0), e isso tem uma consequência direta sobre o resultado do gprof. O flat profile obtido foi:

```
99,90% 29,90s main 0,10% 0,03s partial_pivot (2500 chamadas)
```

A função main concentra 99,90% do tempo; não porque o hotspot seja ambíguo, mas porque o compilador, sob -O2, decidiu inlinear (copiar o corpo de) a função de eliminação inteira dentro de main, já que ela era chamada apenas uma vez. A função gauss_jordan_row_oriented, tecnicamente, deixou de existir como entidade separada no binário final. O call graph confirma essa hierarquia simplificada: main chama partial_pivot exatamente 2500 vezes (uma por iteração k), sem ciclos ou chamadas inesperadas.

4.2 perf stat: Métricas de Hardware

Métrica	Valor
cycles	114,96 bilhões
instructions	250,68 bilhões
IPC	2,18
cache-references	2,20 bilhões
cache-misses	865,81 milhões (39,4%)
branches	31,39 bilhões
branch-misses	7,99 milhões (0,025%)
L1-dcache-load-misses	7,87 bilhões
LLC-load-misses	não suportado pela PMU deste hardware

Tabela 2; perf stat, C Serial, N=2500.

A taxa de cache-miss de 39,4% já é um indicador relevante: mais de um terço dos acessos à cache de dados não encontram o dado buscado, forçando acesso a níveis de memória mais lentos. Este número é a base de comparação para a investigação de causa raiz da Seção 6.3. A taxa de branch-miss extremamente baixa (0,025%) indica que o padrão de desvios do algoritmo (laços com número fixo de iterações) é altamente previsível pelo hardware.

4.3 perf record: Confirmação Independente

O perf report --stdio, que usa amostragem estatística baseada em símbolos do binário (mecanismo distinto da instrumentação do gprof), confirma a mesma conclusão por um caminho independente: main concentra 88,75% do tempo (self time). A função asm_exc_nmi aparece com 9,75%, mas corresponde ao overhead do próprio mecanismo de amostragem do perf, não ao algoritmo em si. A convergência de duas ferramentas com mecanismos de medição completamente diferentes é evidência de que o resultado é real, não um artefato de uma ferramenta específica.

4.4 Valgrind: Callgrind e Cachegrind

O Callgrind, executado com N=250 (reduzido, por ser uma ferramenta de simulação com overhead de aproximadamente 10× em relação à execução nativa), reproduziu a mesma distribuição observada no gprof: main concentra 99,70% das instruções simuladas (lr), com partial_pivot em 0,10%. Essa concordância entre duas ferramentas; uma baseada em contadores reais de hardware (perf/gprof) e outra em simulação completa de CPU (Callgrind); reforça a robustez da conclusão sobre o efeito de inlining.

O Cachegrind, com simulação de cache habilitada (--cache-sim=yes), reportou taxa de miss de leitura na D1 de 6,32%, mas na LL de apenas 0,026%; quase todos os misses de D1 são resolvidos pela LL. Isso ocorre porque, com N=250, a matriz aumentada (~1 MB) cabe folgadoamente na LL simulada (8 MB), mascarando o problema real de banda de memória que só se manifesta com N=2500 (~100 MB, muito maior que qualquer cache real), como evidenciado pelo perf em hardware real (Tabela 2). A redução de N necessária para viabilizar o Callgrind/Cachegrind em tempo hábil subestima, portanto, o efeito de cache observado em escala real.

4.5 strace: Confirmação da Classificação CPU-bound

Syscall	% do tempo em kernel
read	50,39%
write	27,54%
munmap	21,75%

Tabela 3; strace -c, top-3 syscalls, C Serial. Tempo total em kernel: 0,050280 s (0,16% do tempo total de execução).

5. Profiling; Python Serial

5.1 cProfile: Granularidade Preservada

Diferentemente do gprof sob -O2 (Seção 4.1), o cProfile do Python Serial preserva a granularidade completa das funções:

```
ncalls  tottime  cumtime  function    1    1805,02  1806,65
gauss_jordan_serial 2500      1,30    1,64  partial_pivot
```

A função `gauss_jordan_serial` aparece isolada, concentrando 99,91% do tempo de eliminação, com nome próprio preservado. Essa diferença estrutural em relação ao C não reflete uma vantagem do cProfile como ferramenta; decorre de uma diferença fundamental entre compilação e interpretação: o CPython nunca funde duas funções do usuário em uma só (inlining é uma otimização de compilador; o interpretador executa bytecode função por função, sempre preservando os limites definidos no código-fonte).

5.2 perf stat: Comparação Direta com C

Métrica	C Serial	Python Serial	Razão
Wall-clock	29,82 s	31min 45s	63,9×
instructions	250,68 bilhões	20,14 trilhões	80,3×
IPC	2,18	2,90	Python maior
Taxa de cache-miss	39,4%	28,2%	Python menor

Tabela 4; Comparação C vs. Python Serial, perf stat completo.

Este resultado apresenta uma aparente contradição que precisa ser interpretada com cuidado: o Python tem IPC maior (2,90 contra 2,18) e taxa de cache-miss menor (28,2% contra 39,4%) que o C; dois indicadores que, isolados, sugeririam maior eficiência de hardware; e, ainda assim, é 63,9 vezes mais lento em tempo total.

A explicação é que eficiência por ciclo de CPU não é sinônimo de eficiência total: o gargalo do Python está no volume absoluto de instruções, não na qualidade da execução de cada uma pelo hardware. Para realizar a mesma operação que o C executa com poucas instruções de máquina, o interpretador CPython verifica o tipo de cada operando, segue ponteiros de objeto e atualiza contadores de referência (mecanismo também responsável pela existência do GIL, Seção 7.1) antes de calcular. Isso multiplica o número de instruções por 80,3×; como essas instruções pertencem ao próprio código do interpretador, compacto e otimizado, elas ainda são executadas com boa eficiência de cache e IPC, o que explica por que essas duas métricas isoladamente favorecem o Python.

5.3 strace: Overhead de Inicialização do Interpretador

Indicador	C Serial	Python Serial
Chamadas de mmap	15	608
Chamadas de brk	poucas (não dominante)	1.581

Tempo total em kernel	0,050 s (0,16% do total)	0,068 s (0,0038% do total)
-----------------------	--------------------------	----------------------------

Tabela 5; strace, comparação de startup e alocação de memória.

As 608 chamadas de mmap no Python (40× mais que o C) refletem o custo de inicialização do interpretador: carregamento do executável, mapeamento da libpython e importação de módulos. As 1.581 chamadas de brk (ajuste de heap) refletem o alocador do CPython solicitando memória repetidamente, consequência do grande volume de pequenos objetos criados durante a execução; o mesmo fenômeno de boxing detalhado na Seção 7.3. Proporcionalmente, o tempo em kernel é ainda menor que no C (0,0038% vs. 0,16%), reforçando a classificação CPU-bound.

6. Profiling; C + OpenMP

6.1 Escalabilidade: Speedup e Eficiência

Threads	Wall-clock	Speedup	Eficiência
1	28,70 s	1,04×	103,9%
2	25,18 s	1,18×	59,2%
4	24,44 s	1,22×	30,5%
8	25,79 s	1,16×	14,5%

Tabela 6; Speedup e eficiência, C + OpenMP, 1/2/4/8 threads (baseline: C Serial, 29,82 s).

6.2 Comparação com a Lei de Amdahl

A Lei de Amdahl estabelece que o speedup máximo alcançável é limitado pela fração inerentemente serial do programa (s), segundo $\text{Speedup}(n) = 1 / (s + (1-s)/n)$. Essa fração foi medida diretamente com o Callgrind (Seção 4.4): a função main (que engloba a eliminação, paralelizável) concentra 99,70% do tempo, restando $s = 0,30\%$ para as etapas seriais (leitura de arquivo, montagem da matriz aumentada, escrita do resultado). Com esse valor de s , a Lei de Amdahl prevê speedup de 7,83× para 8 threads.

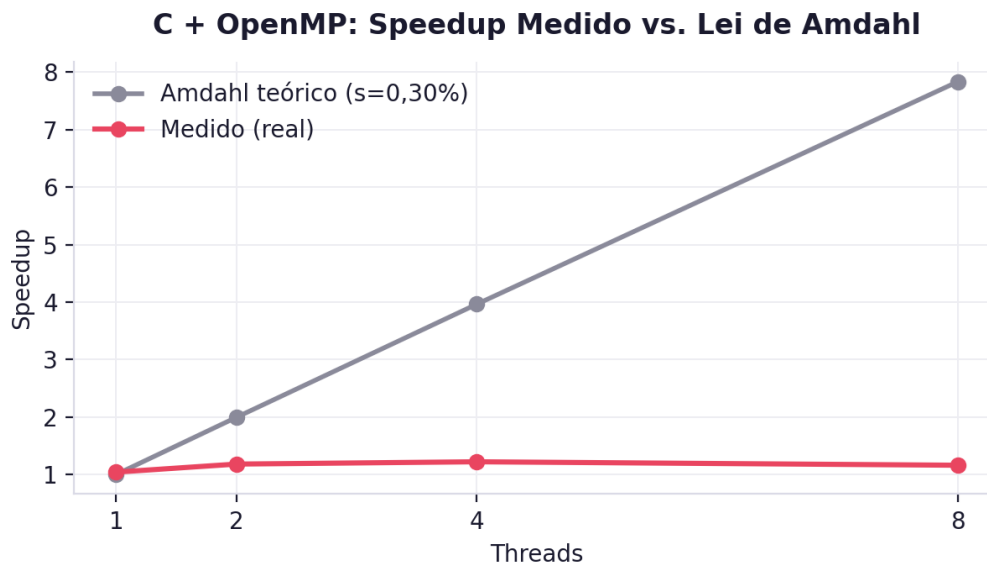


Figura 1; Speedup medido vs. speedup teórico previsto pela Lei de Amdahl ($s = 0,30\%$).

O speedup medido (1,16× em 8 threads) fica drasticamente abaixo do previsto (7,83×). Essa divergência de mais de 6× não pode ser explicada pela Lei de Amdahl, que assume implicitamente que a parte paralelizável tem acesso a capacidade de cálculo proporcional ao número de threads. Existe um segundo fator limitante, não modelado por Amdahl, investigado na próxima seção.

6.3 Causa Raiz: Algoritmo Memory-Bound e Efeito de Hyper-Threading

A causa raiz da divergência acima foi investigada com perf stat, variando o número de threads e

observando IPC e taxa de cache-miss:

Threads	IPC	Taxa de cache-miss	Context switches (vol.+invol.)
1	2,310	40,9%	483
2	1,319	40,4%	308
4	0,703	44,7%	831
8	0,355	51,7%	18.347

Tabela 7; IPC, taxa de cache-miss e context switches por número de threads, C + OpenMP.

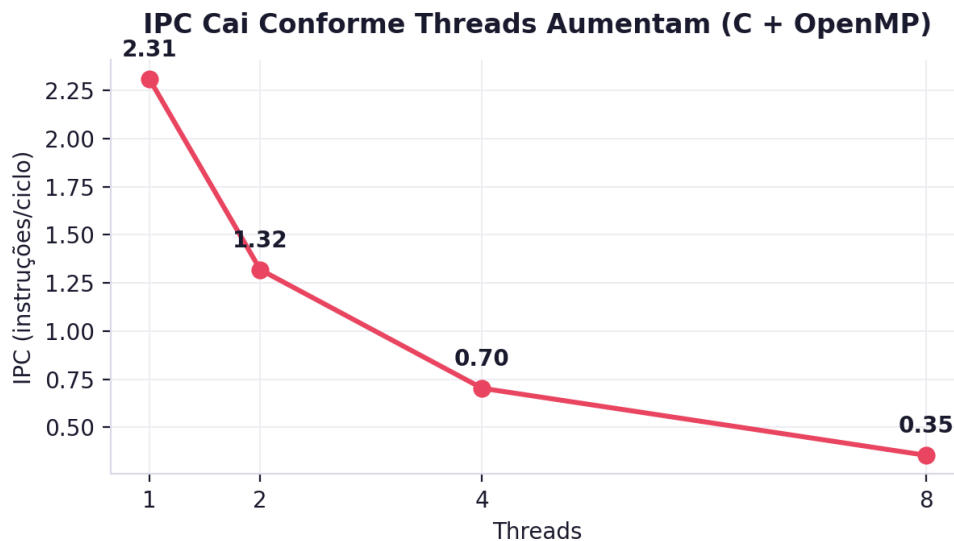


Figura 2; Queda de IPC conforme o número de threads aumenta.

A explicação da degradação ocorre em duas etapas distintas e cumulativas:

Etapas 1: Saturação de banda de memória (já visível de 1 para 4 threads)

O algoritmo tem intensidade aritmética muito baixa (Seção 2.1): duas leituras e uma escrita de memória para apenas duas operações de ponto flutuante, o que o classifica como memory-bound; o fator limitante não é a capacidade de cálculo da CPU, mas a velocidade de chegada dos dados da RAM. Quando threads adicionais competem pelo mesmo barramento de memória, de capacidade fixa, o IPC já cai 70% entre 1 e 4 threads (de 2,310 para 0,703), antes mesmo de qualquer efeito de Hyper-Threading, já que 4 threads correspondem a 4 núcleos físicos distintos.

Etapas 2: Contenção de Hyper-Threading (agravamento de 4 para 8 threads)

O processador possui apenas 4 núcleos físicos; as 8 threads lógicas formam 4 pares que compartilham fisicamente a mesma FPU e as mesmas caches L1/L2 (mapeamento da Seção 1.3). Dentro de um núcleo físico, as duas threads lógicas não executam ponto flutuante verdadeiramente em paralelo; dividem as mesmas unidades de execução. O salto de 22× nos context switches (de 831 para 18.347) é a evidência direta dessa disputa adicional.

6.4 perf record: Overhead das Diretivas OpenMP

O perf record --stdio com OMP_NUM_THREADS=4 identificou a função `gauss_jordan_row_oriented._omp_fn.0`; a região paralela extraída pelo compilador para ser despachada pelo runtime do OpenMP; concentrando 92,12% do tempo (self). Diferentemente do C Serial, esta função não foi absorvida por inlining: regiões paralelas OpenMP não podem ser inlineadas, pois o runtime (libgomp) precisa de um ponteiro de função válido para despachar a execução às threads.

O overhead agregado de todas as funções pertencentes à biblioteca libgomp.so (incluindo GOMP_parallel e mecanismos internos de sincronização/distribuição de trabalho do `schedule(static)`) somou apenas 3,44% do tempo total. Este resultado é relevante: o mecanismo de paralelização do OpenMP em si é eficiente; o overhead de gerenciar a região paralela é pequeno. O gargalo de desempenho documentado nesta seção está integralmente dentro do código de eliminação (dentro de `_omp_fn.0`), não na infraestrutura de paralelização.

6.5 Caminhos Possíveis de Mitigação (Não Implementados)

A investigação de causa raiz aponta para estratégias conhecidas na literatura que poderiam mitigar o problema de banda de memória, embora nenhuma tenha sido implementada nesta entrega, por estar fora do escopo do trabalho:

- Blocking/tiling: reorganizar o algoritmo para reaproveitar um bloco de dados já presente em cache antes de descartá-lo, aumentando a intensidade aritmética efetiva;
- Redução de precisão: usar float em vez de double, reduzindo pela metade o volume de bytes transferidos por elemento;
- Decomposição LU em blocos com pipeline, técnica descrita por Michailidis e Margaritis (2011).

7. Profiling; Python Paralelo: Threading e Multiprocessing

7.1 O Mecanismo do GIL

O CPython gerencia memória por contagem de referências: cada objeto carrega um contador de quantas variáveis apontam para ele, usado para liberá-lo quando chega a zero. Essa contagem não é segura para ser alterada por múltiplas threads sem proteção, e a solução do CPython é um único bloqueio global (o GIL) que deve ser mantido por qualquer thread para executar bytecode Python; apenas uma thread por vez interpreta instruções, independentemente do número de núcleos físicos disponíveis.

7.2 Evidência do GIL: Threading

Threads	Wall-clock	Speedup vs. serial	%CPU
1	1861,94 s	1,02×	99%
2	4207,00 s	0,45×	100%
4	4301,70 s	0,44×	101%
8	4358,38 s	0,44×	102%

Tabela 8; Speedup e %CPU, Python Threading (baseline: Python Serial, 1904,98 s).

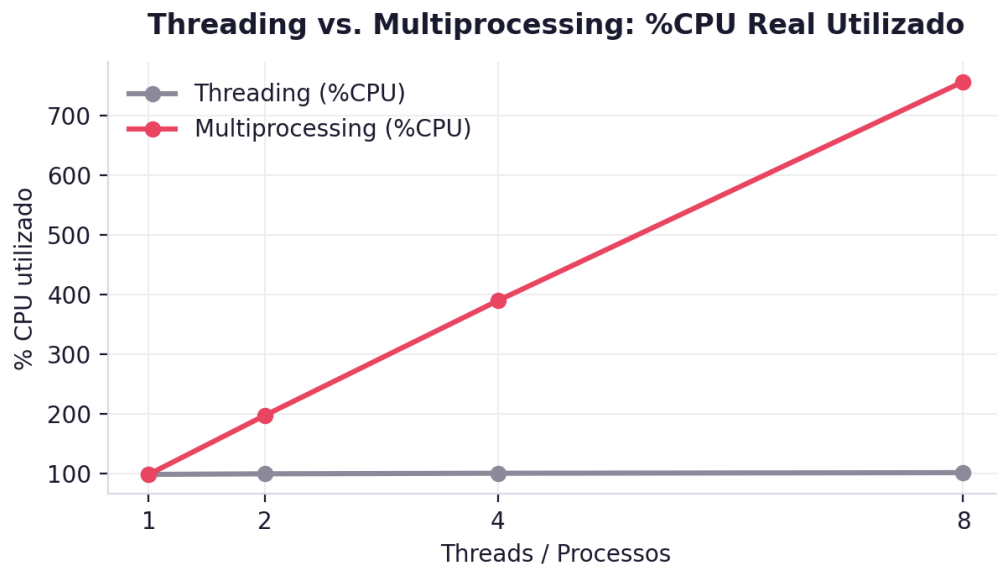


Figura 3; %CPU real utilizado: Threading permanece próximo de 100% em qualquer configuração; Multiprocessing escala genuinamente.

O %CPU nunca ultrapassa 102%, mesmo com 8 threads solicitadas; o sistema operacional nunca registra mais do que o equivalente a um núcleo físico sendo utilizado, prova direta e inequívoca de que não há paralelismo real de hardware ocorrendo, independentemente de quantas threads sejam criadas.

O cProfile revelou onde o tempo é consumido: a função `Thread.join()` apresentou tempo cumulativo de 8375,53 segundos; 195% do tempo total de execução (valor superior a 100% porque múltiplas threads

se revezam esperando a liberação da mesma trava, e os tempos de espera se sobrepõem na contabilização cumulativa). A função `threading.py:start (Thread.start())` foi chamada 10.000 vezes, com tempo cumulativo de 1222,83 segundos; e o método `acquire` de `_thread.lock`; o mecanismo de baixo nível por trás de `Lock.acquire()` e da disputa pelo GIL; foi chamado 70.000/50.000 vezes, somando 9591,74 segundos cumulativos. Apenas 34,5% do tempo total correspondeu à função de trabalho real (`update_chunk`); o restante é overhead de criação, destruição e sincronização de threads.

O `perf record --stdio`, com call graph completo, identificou `_PyEval_EvalFrameDefault` (o núcleo de avaliação de bytecode do interpretador) concentrando 60,94% do tempo `self`; consistente com a explicação da Seção 5.2 sobre o volume de instruções do interpretador. Funções específicas de sincronização do GIL (`PyThread_acquire_lock`, `sem_wait`) apareceram com menos de 0,02% do tempo; resultado que exige interpretação cuidadosa, discutida a seguir.

Achado Metodológico: Amostragem vs. Instrumentação

A ausência de `PyThread_acquire_lock` no `perf record` não significa ausência de disputa pelo GIL; significa que cada evento de espera pela trava dura apenas microssegundos, e a amostragem estatística do `perf` (que captura o estado da CPU em intervalos periódicos) raramente coincide exatamente com um desses eventos breves, mesmo com milhões de amostras coletadas. O `cProfile`, por instrumentar cada chamada de função individualmente (sem amostrar), capturou a disputa pelo GIL de forma completa e inequívoca através do tempo cumulativo de `Thread.join()`.

7.3 Evidência de Paralelismo Real: Multiprocessing

Processos	Wall-clock	Speedup vs. serial	%CPU
1	5912,32 s	0,32×	99%
2	2925,46 s	0,65×	198%
4	1971,36 s	0,97×	390%
8	2284,17 s	0,83×	757%

Tabela 9: Speedup e %CPU, Python Multiprocessing (baseline: Python Serial, 1904,98 s).

Diferentemente do Threading, cada processo criado por multiprocessing possui seu próprio interpretador Python completo, com seu próprio GIL independente; não há disputa entre processos pela mesma trava. O %CPU confirma isso de forma direta: escala de 99% (1 processo) até 757% (8 processos), e o próprio `perf stat` reporta "4.0 CPUs_utilized" e "7.6 CPUs_utilized" para 4 e 8 processos, respectivamente; paralelismo genuíno, sem ambiguidade.

O `cProfile` mostrou a função `posix.read` (leitura do pipe de comunicação entre processos) como dominante, com 3269,37 segundos de tempo cumulativo, refletindo o custo real desta versão: comunicação entre processos isolados, não disputa por trava. A função `Pool.starmap` (equivalente, nesta implementação, ao despacho de tarefas feito tipicamente por `Process.start/Pool.map`) somou 3684,42 segundos cumulativos, e as funções internas de gerenciamento do Pool (`_handle_tasks` e `_handle_results`) somaram aproximadamente 2056 segundos. Notavelmente, `pickle.dumps/pickle.loads`; mecanismo padrão de serialização do módulo multiprocessing, citado no enunciado como ponto de observação; não aparece entre as funções mais custosas, confirmando que a decisão de projeto de usar multiprocessing.shared_memory para a matriz (evitando serializar o dado grande a cada iteração) foi eficaz; apenas os parâmetros pequenos de cada tarefa (início, fim, k) são de fato serializados através do

mecanismo interno de fila (Queue) usado pelo Pool para despachar tarefas.

O Custo do Boxing

Uma observação adicional, relevante para o item de análise de overhead de processo: o Maximum RSS do Multiprocessing (592,2 MB) é 35% menor que o do Threading (919,6 MB), apesar de realizar o mesmo trabalho. Isso ocorre porque a memória compartilhada armazena os números como bytes brutos (8 bytes por double), enquanto listas Python armazenam cada número como um objeto completo, com overhead de 24 a 28 bytes por objeto (cabeçalho de tipo, contador de referência). Em contrapartida, cada acesso a um valor na memória compartilhada exige boxing; a criação de um novo objeto float na hora da leitura, já que memória compartilhada não pode armazenar objetos Python diretamente.

Comparação de Três Vias: Serial, Threading e Multiprocessing

Versão	Instructions	IPC	Wall-clock
Python Serial	20,14 trilhões	2,90	31min 45s
Python Threading (4t)	20,14 trilhões	2,70	1h 11min 42s
Python Multiprocessing (4p)	55,79 trilhões	2,50	32min 51s

Tabela 9b; Comparação de instructions, IPC e wall-clock entre as três versões Python.

Esta comparação de três vias confirma parcialmente a expectativa geral descrita no enunciado. O número de instruções do Multiprocessing (55,79 trilhões) é de fato maior que o do Serial; 2,77 vezes mais, consistente com o overhead de manter 4 interpretadores CPython completos rodando simultaneamente. Já o Threading apresenta número de instruções praticamente idêntico ao Serial (20,14 trilhões, razão de 1,00×), porque, como discutido na Seção 7.1, apenas uma thread executa bytecode por vez independentemente de quantas sejam criadas; não há "trabalho extra" de múltiplos interpretadores, apenas o overhead de sincronização já quantificado.

Quanto ao wall-clock, a expectativa geral de que "o tempo total deve ser menor que o serial, apesar do maior número de instruções, por conta do paralelismo real" não se confirma nesta implementação específica: o Multiprocessing com 4 processos (32min 51s) é, na verdade, ligeiramente mais lento que o Python Serial (31min 45s); um resultado que poderia parecer contraditório com o paralelismo genuíno já demonstrado (%CPU de 390%, Seção 7.3). A explicação, detalhada na Seção 8.1, é que o overhead estrutural de boxing ao acessar a memória compartilhada consome praticamente todo o ganho do paralelismo real quando a base de comparação é o Python Serial comum, e não a própria versão de 1 processo. Este é um resultado relevante: confirma que paralelismo real (evidenciado pelo IPC e %CPU) não garante, por si só, redução de tempo total quando a estrutura de dados subjacente introduz overhead suficiente.

Custo Fixo de Criação de Processos vs. Granularidade dos Dados

Criar um processo via `fork()` (duplicar tabelas de página, inicializar um novo interpretador) custa ordens de grandeza mais que criar uma thread, e esse custo é fixo por processo, independentemente do volume de trabalho que ele executará. Neste algoritmo, o Pool é criado uma única vez e reaproveitado ao longo das 2500 iterações, amortizando bem esse custo fixo: cada tarefa despachada processa uma fatia substancial (até 625 linhas, para $N=2500$ e 4 processos), tornando o tempo útil por tarefa muito maior que o overhead de comunicação. Com granularidade muito menor (por exemplo, um processo

por elemento da matriz), esse mesmo custo fixo dominaria o tempo total, tornando o paralelismo contraproducente independentemente do número de processos disponíveis.

7.4 Limitação Metodológica: Volume de Dados de Profiling

O `perf record --stdio -g` da versão Multiprocessing (4 processos) gerou um arquivo de aproximadamente 9 GB; ordens de magnitude maior que os arquivos equivalentes das demais versões. A causa é que a opção de call graph completo faz o `perf` desenrolar a pilha de chamadas a cada amostra, e o multiprocessing cria um processo do sistema operacional por trabalhador (diferente de threads, que compartilham um único processo), multiplicando o número de fluxos de execução simultaneamente amostrados. O processamento deste arquivo com `perf report` tornou-se computacionalmente impraticável em tempo hábil, mesmo com opções de redução de carga (`-g none, --percent-limit`).

7.5 Alternativas ao GIL Além do Multiprocessing

Além de recorrer a processos separados, duas outras estratégias permitem contornar a limitação do GIL para trechos específicos de código, sem abandonar o modelo de threads:

- Cython com blocos `nogil`: permite liberar o GIL manualmente em regiões de código que não manipulam objetos Python (por exemplo, operações sobre arrays de tipos nativos), permitindo que outras threads Python executem durante esse intervalo;
- Extensões C (via `ctypes` ou `ctypes`): chamadas a funções C nativas liberam o GIL durante a execução da função, e podem executar em paralelo real com outras threads Python.

O custo comum às duas abordagens é a necessidade de reescrever a parte crítica do código fora do Python puro; uma opção que altera fundamentalmente a natureza da implementação em relação a permanecer inteiramente em Python.

8. Comparação dos 3 Modelos de Paralelização

A Tabela 10 consolida os três modelos de paralelização estudados (OpenMP, Threading e Multiprocessing), na configuração de 4 threads/processos, incluindo características estruturais de cada modelo além das métricas de desempenho.

Modelo	Speedup	Eficiência	Wall-clock	Max RSS	Memória	Comunicação	Criação
OpenMP (C)	1,22×	30,5%	24,44 s	192,9 MB	Compartilhada	Direta	Baixo (thread)
Python Threading	0,44×	11,1%	1h 11min 42s	919,6 MB	Compartilhada (GIL)	Direta (GIL)	Baixo (thread)
Python Multiproc.	0,97×	24,2%	32min 51s	592,2 MB	Isolada (IPC)	Serialização*	Alto (fork)

Tabela 10; Comparação dos 3 modelos de paralelização, 4 threads/processos. * Na implementação deste trabalho, mitigada via `shared_memory` para o dado grande (Seção 7.3); "Alto (fork)" inclui também duplicação parcial do interpretador por processo.

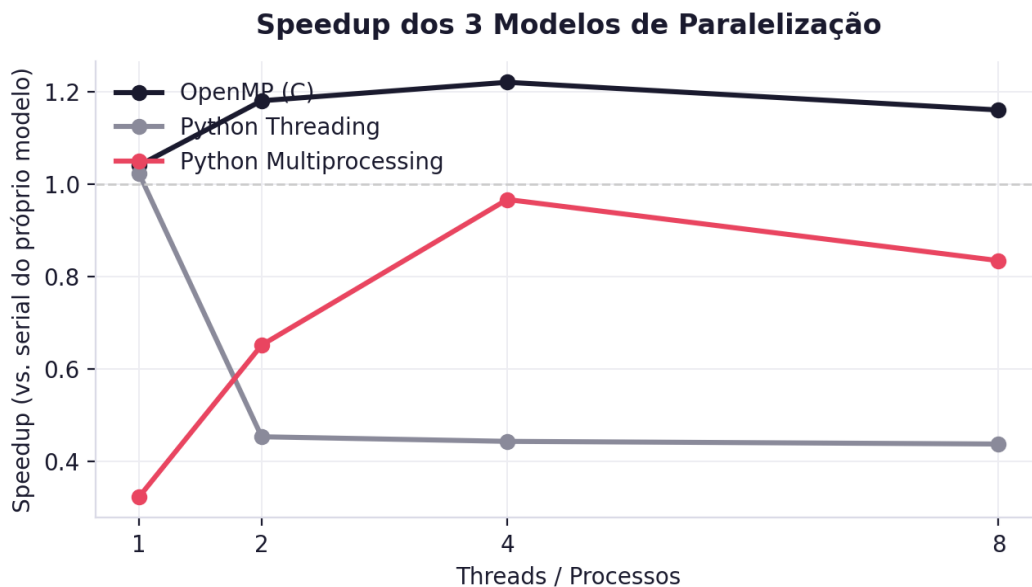


Figura 4; Speedup dos 3 modelos de paralelização, 1/2/4/8 threads/processos, cada um comparado ao seu próprio baseline serial.

8.1 Detalhe Interessante Desta Comparação

A Tabela 10 mostra o Multiprocessing (0,97×) abaixo do OpenMP (1,22×); resultado que, à primeira vista, parece contradizer o paralelismo real demonstrado na Seção 7.3 (escalabilidade de 3,00× entre 1 e 4 processos). Não há contradição: a diferença está na base de comparação. O valor de 3,00× isola o efeito do paralelismo, comparando a versão de 4 processos apenas contra a mesma versão com 1 processo. O valor de 0,97× desta tabela compara contra o Python Serial comum; o ponto de partida real de qualquer implementação. O overhead estrutural introduzido pela escolha de memória compartilhada (o custo de boxing discutido na Seção 7.3) consome quase a totalidade do ganho de paralelismo quando medido a partir desse ponto de partida realista.

Portanto paralelizar uma implementação cuja estrutura de dados já introduziu overhead adicional não necessariamente supera uma implementação serial mais simples. Paralelismo e eficiência de acesso a dados são dimensões distintas do problema de desempenho, e otimizar apenas uma delas não garante ganho líquido.

9. Compilado vs. Interpretado: Síntese Comparativa C × Python

Esta seção reúne, de forma consolidada, todas as comparações diretas entre a linguagem compilada (C) e a interpretada (Python) realizadas ao longo deste trabalho.

9.1 Tabela Consolidada

Métrica	C (serial)	Python (serial)	Razão / Observação
Wall-clock	29,82 s	31min 45s	63,9× mais lento
Instructions	250,68 bilhões	20,14 trilhões	80,3× mais instruções
IPC	2,18	2,90	Python maior (contraintuitivo)
Taxa de cache-miss	39,4%	28,2%	Python menor (contraintuitivo)
Granularidade de profiling	Colapsada por inlining (-O2)	Preservada (sem inlining)	Diferença estrutural
Maximum RSS (mesmo trabalho)	192,6 MB	918,5 MB	4,77× mais memória
Chamadas de mmap (startup)	15	608	40× mais

Tabela 11; Síntese consolidada de todas as comparações C vs. Python realizadas.

9.2 Discussão: Implicações para HPC (*High Performance Computing*)

O achado mais importante desta síntese (já introduzido na Seção 5.2) é que as métricas de eficiência por ciclo (IPC, cache-miss) favorecem o Python, e ainda assim ele é 63,9× mais lento: o que determina o tempo de execução é o produto entre número de instruções e tempo médio de cada uma, e é no primeiro fator (80,3× maior) que reside o custo real do interpretador.

Isso tem implicação direta para HPC: otimizações de baixo nível (localidade de cache, branch prediction) têm efeito limitado quando a linguagem já introduz um multiplicador de instruções da ordem de 80× antes delas entrarem em jogo. Para código numericamente intensivo como o estudado aqui, a escolha de linguagem compilada é, isoladamente, mais determinante para o desempenho final do que qualquer paralelização aplicada depois; um paralelismo perfeito de 8× em Python ainda deixaria o programa mais lento que o C serial sem paralelizar.

10. Conclusões

Os principais achados são:

- O overhead do interpretador Python, medido diretamente, corresponde a 80,3× mais instruções de máquina e 63,9× mais tempo de execução que o C compilado com -O2; apesar de o Python apresentar, isoladamente, IPC maior e taxa de cache-miss menor, demonstrando que eficiência por ciclo não equivale a eficiência total.
- O C + OpenMP não escala conforme a Lei de Amdahl prevê (7,83× teórico contra 1,16-1,22× medido) porque o algoritmo é memory-bound: a baixa intensidade aritmética satura a banda de memória compartilhada já com poucas threads, e a contenção de Hyper-Threading (threads lógicas disputando a mesma unidade física) agrava ainda mais o problema entre 4 e 8 threads.
- O Python Threading nunca demonstra ganho de paralelismo em nenhuma configuração testada (%CPU nunca ultrapassa 102%), porque o GIL impede a execução simultânea de bytecode por mais de uma thread; uma barreira de software que atua antes mesmo de qualquer disputa por recursos físicos de hardware.
- O Python Multiprocessing demonstra paralelismo real (%CPU até 757%, confirmado por perf stat), mas o overhead estrutural de sua implementação (custo de boxing ao acessar memória compartilhada) consome quase a totalidade do ganho quando comparado ao Python Serial comum, evidenciando que paralelismo e eficiência de estrutura de dados são dimensões independentes do problema de desempenho.
- Em nenhuma das três estratégias de paralelização testadas o speedup superou 1,22×, evidenciando que, para este algoritmo e hardware, a arquitetura física; não a linguagem ou o modelo de paralelização escolhido; é o fator limitante dominante.

10.1 Limitações do Trabalho

- A fração serial (s) usada na comparação com a Lei de Amdahl foi medida via Callgrind com N reduzido ($N=250$, necessário por limitação de tempo de execução da ferramenta), podendo divergir marginalmente do valor real em $N=2500$;
- O perf record da versão Multiprocessing não pôde ser processado devido ao volume de dados gerado (~9 GB), como discutido na Seção 7.4; a análise desta versão apoiou-se em perf stat e cProfile;

10.2 Referências

ALASSAFI, Madini O.; ALSENANI, Yousef S. Inverse matrix using Gauss elimination method by OpenMP. International Journal of Information Technology and Computer Science, [S.l.], v. 8, n. 2, p. 41-46, fev. 2016.

MICHAILIDIS, Panagiotis D.; MARGARITIS, Konstantinos G. Open multi processing (OpenMP) of Gauss-Jordan method for solving system of linear equations. In: IEEE INTERNATIONAL CONFERENCE ON COMPUTER AND INFORMATION TECHNOLOGY, 11., 2011. Proceedings [...]. [S.l.]: IEEE, 2011. p. 314-319.